

DiviSense AI

Architecture & Design Document

Agentic dividend forecasting for NSE-listed Indian companies

Version 1.0 (MVP) | July 10, 2026

Repository: github.com/smahendra2601/divisense-ai

This document explains, in simple language, how DiviSense AI works from the moment a user types a question to the moment an answer appears, why each design decision was made, and how the system behaves when things go wrong — every scenario described here was actually executed and verified, not just designed on paper.

1. What is DiviSense AI?

DiviSense AI answers one kind of question really well: **"What will this Indian company do with its dividend?"** You can type a stock ticker (like ITC) or a plain-English question (like *"Will Infosys increase its dividend next quarter?"*), and the system fetches live market data, computes financial health metrics, and produces a forecast — always with its reasoning shown, a confidence level, and a clear disclaimer that this is research, not advice.

It handles three kinds of requests:

- **Forecast** — "ITC" or "Forecast Coal India's dividend for next year" produces a forecast of next fiscal year's total dividend per share, as a range in rupees, with the likely interim/final split and timing window.
- **Direct question** — "Will Infosys increase its dividend next quarter?" gets a direct answer first (likely / unlikely / unclear), then the supporting forecast and metrics.
- **Everything else** — multi-company screeners ("top dividend payers") get a polite "out of scope" explanation, and unclear input gets a friendly request to rephrase, with examples.

Three hard constraints shaped the whole design: it must run on an ordinary laptop (no cloud servers), it must use only **free-tier** AI APIs (which have strict usage limits), and the first version had a three-day deadline. Almost every design choice below traces back to one of these.

2. The Big Picture: Four Tiers

The system is organised as four layers, each with one job. Data flows up: raw market data at the bottom becomes computed metrics, then reasoned forecasts, then a friendly screen at the top.

Tier	Name	What it does	Main files
4	Presentation	The Streamlit web page and the command-line tool. Takes the question, shows progress, renders the answer.	app.py, forecast.py
3	Orchestration	The 'brain'. A LangGraph workflow that decides what the user wants, calls each service in order, checks the answer, and assembles the report.	src/graph.py, src/intent.py, src/report.py
2	Intelligence services	Three independent services: a calculator for financial ratios (pure code, no AI), a document-search index over annual reports, and a router that talks to the AI models.	src/ratio_engine.py, src/rag.py, src/llm_router.py
1	Data acquisition	Everything that touches the outside world: live Yahoo Finance fetches, company-name lookup, corporate-action records, the disk cache, and one-time PDF ingestion.	src/data_agent.py, src/ticker_map.py, src/corp_actions.py, src/cache.py, src/pdf_ingest.py

A note on vocabulary: an **LLM** (large language model) is the AI that reads and writes text — we use two, Groq's Llama 3.3 and Google's Gemini. **RAG** (retrieval-augmented generation) means searching a company's annual report for relevant paragraphs and handing them to the LLM as context. A **ticker** is a stock's short code on the NSE exchange, like INFY for Infosys.

3. The Journey of One Question

Here is exactly what happens when a user asks *"Will Infosys increase its dividend next quarter?"* — seven steps, at most three AI calls:

Step	Node	What happens
1	Intent	One small LLM call reads the question and extracts: this is a dividend question (not a screener), the company mentioned is 'Infosys', the horizon is 'next quarter'. The LLM only proposes the company NAME — a lookup table (ticker_map) makes the final call that Infosys = INFY. If the user had typed just 'ITC', a regex spots it and this LLM call is skipped entirely.
2	Data	Live fetch from Yahoo Finance: full dividend history, current price, and the last four years of income statements, balance sheets and cash-flow statements. Results are cached on disk for one hour.
3	Ratio	Pure Python/pandas — no AI — computes ten metrics: current yield, payout ratio, 5-year dividend growth rate, free-cash-flow coverage, consecutive-increase streak, a 0-100 consistency score, debt trend, and the recent trajectory (rising / flat / falling / mixed).
4	RAG	Searches the local annual-report index for paragraphs about dividend policy and capital allocation. If no reports were ever ingested for this company, it simply returns nothing and the pipeline continues.
5	Forecast	The second LLM call. It receives the metrics as JSON, any report snippets, and the user's exact question. Strict rules: answer the question FIRST and directly; use ONLY numbers present in the metrics; and apply the 'Indian cadence rule' — NSE companies pay interim + final dividends, not US-style quarterly, so 'next quarter' is mapped to the actual payout calendar and this is said explicitly.
6	Critic	The third LLM call plays devil's advocate: does every number in the forecast trace back to the metrics? Is 'likely increase' consistent with a falling trajectory? If it rejects and the call budget allows, the forecast is redone once with the critique attached; otherwise the answer is shown with confidence downgraded to LOW and the objections visible.
7	Report	Pure code assembles the final answer: direct-answer banner, forecast card, metrics table, corporate actions, dividend-history chart (in the web app), 'data as of' timestamp, an expandable agent trace, and the disclaimer. Clarify / out-of-scope / error cases get friendly messages instead.

If anything fails at any step — a bad ticker, Yahoo down, both AI providers unavailable — the flow short-circuits to the Report step, which explains the problem in plain words. The user never sees a stack trace.

4. Design Choices and the Reasons Behind Them

4.1 The AI never does arithmetic

Choice: every financial number is computed by ordinary code (the ratio engine); the LLM only interprets those numbers. **Why:** language models are excellent at reasoning in words and notoriously unreliable at arithmetic and at recalling financial figures — they will confidently invent a payout ratio. By computing everything deterministically and instructing the model to use only the provided values, the numbers in every report are exactly as trustworthy as the source data. The Critic agent then double-checks that the model obeyed. In our audited live runs, an automated cross-check of every number the model cited against the computed metrics found **zero fabricated figures**.

4.2 The AI never picks the ticker

Choice: the LLM proposes a company *name*; a CSV lookup table (about 115 aliases covering the Nifty 50 and major PSU dividend payers) plus a Yahoo Finance existence check make the final decision. **Why:** a hallucinated ticker is the worst possible failure — the user asks about Infosys and silently gets numbers for a different company. Name-to-ticker mapping is a solved, deterministic problem, so we never delegate it to a probabilistic model. If the name cannot be resolved, the system asks the user to clarify, echoing back the name it tried.

4.3 Fetch-on-demand with a one-hour cache

Choice: no database of market data; every query fetches live from Yahoo Finance, cached on disk for one hour. **Why:** a pre-loaded database is stale the day after you build it, and maintaining it is work the three-day budget did not have. Live fetches guarantee freshness by design; the one-hour cache protects a demo session (and Yahoo's goodwill) from repeated identical fetches. Annual report PDFs are the one exception — they are static documents, so pre-indexing them once does not violate freshness.

4.4 Two AI providers with automatic fallback

Choice: Groq (fast Llama 3.3) is primary; Google Gemini is the fallback, and the router switches automatically on any failure — rate limits, invalid keys, outages. **Why:** free tiers are unreliable by nature: Groq allows roughly 30 requests/minute, Gemini about 1,500/day, and either can be down. One provider would make the demo fragile; two make it resilient. This paid off in an unexpected way (see section 6): Google *retired* our pinned Gemini model overnight, and the simulation designed to test fallback is what caught it.

4.5 A hard budget of 3 AI calls per query

Choice: each query may spend at most 3 LLM calls (intent + forecast + critic); a bare ticker skips intent and spends only 2. The critic's one retry is only allowed if it fits in the budget; otherwise the answer ships with confidence downgraded to LOW and the critic's objections shown. **Why:** quota discipline. On free tiers, an uncontrolled retry loop can burn a day's quota in minutes. We initially allowed retry + re-check (up to 5 calls) and our own audit caught the overrun — the fix enforces the ceiling in the workflow's routing logic, and a test asserts the invariant across every scenario. Every response also caches, so repeating a question costs zero calls.

4.6 A critic agent that checks the forecaster

Choice: a second LLM pass reviews the first one's output before the user sees it. **Why:** the cheapest way to catch the classic LLM failure modes — invented numbers, and conclusions that contradict the data (e.g.

'likely increase' when the trajectory is falling and the payout ratio is already stretched). When the critic rejects and no retry budget remains, we deliberately show the answer *with* the objections and a LOW confidence badge rather than hiding the disagreement — transparency over polish.

4.7 Dividends attributed to their results fiscal year

Choice: Yahoo gives dividend *ex-dates*; we map each payout to the fiscal year it was declared *for* (an ex-date in Oct–Dec belongs to the FY ending next March; Jan–Sep to the FY just ended). **Why:** Indian companies pay an interim dividend mid-year and a final dividend months *after* the March year-end. Naive calendar-year grouping splits one fiscal year's dividend across two years and matches nothing the user has ever seen. With this rule, our totals line up with Screener.in (e.g. ITC FY25 = Rs 6.50 interim + Rs 7.85 final = Rs 14.35).

4.8 Detecting a half-collected year

Choice: if the newest fiscal year has fewer payouts than the company's recent norm, it is marked *provisional* and the money metrics use the last complete year instead, with a visible data note. **Why:** in July 2026, Coal India's FY2026 showed only Rs 15.75 because its final dividend had not gone ex-dividend yet — the naive number made a 7.4%-yield stock look like a 3.6% one. The norm is judged against the last five years only, because Coal India's payout rhythm changed over its history and the full-history median hid the problem.

4.9 Document search that is optional by design

Choice: annual-report search (RAG) uses a local embedding model (all-MiniLM-L6-v2) and a local Chroma index, and returns an empty result — never an error — when a company has no documents. **Why:** local embeddings cost zero API quota and work offline. Making RAG strictly optional means ingesting reports improves answers but is never required — essential for an MVP where nobody has time to collect 100 PDFs before the first demo.

4.10 Timeouts and swap-friendly seams

Choice: every external call has an explicit time limit (60s per AI call, 45s for a full market-data fetch) and clear error types; data sources sit behind interfaces (e.g. corporate actions come from a CSV today, and a live NSE scraper can replace it by changing one factory function). **Why:** a hung network socket must never freeze the app, and the roadmap items (live scrapers, new providers, local models) should plug in without touching the rest of the system.

4.11 Transparency everywhere

Choice: every answer carries its reasoning chain, a confidence badge, a 'data as of' timestamp, an expandable agent trace (what each step did, how many AI calls were spent), and a disclaimer. **Why:** this is a research tool making probabilistic statements about money. Showing the work is what separates a defensible forecast from a magic 8-ball, and the trace doubles as the debugging and audit view.

5. Verified Success Scenarios (run live, not hypothetical)

These five queries were executed end-to-end against live Yahoo Finance data and live AI providers, with instrumentation counting every provider call:

Query	Path taken	AI calls	Result
ITC	Bare ticker: regex + lookup, intent LLM skipped	2	Forecast Rs 14.8–16.3/share, medium confidence, critic approved. All cited numbers traced to computed metrics.
Will Infosys increase its dividend next quarter?	Question path: intent, forecast, critic	3	Direct answer applying the cadence rule ('next quarter' mapped to the Oct–Mar interim window). Genuinely uncertain case: critic flagged it, answer shown with LOW confidence and objections visible.
Forecast Coal India's dividend for next year	Question path	3	'Coal India' resolved to COALINDIA by the lookup table. Forecast Rs 32.0–38.0, medium confidence, critic approved first pass. Provisional-FY note shown for the half-collected FY2026.
Top public sector dividend paying companies	Screeners detected	1	Polite out-of-scope card: single-company tool, screeners are on the roadmap; examples of valid queries offered.
ZZZZZ	Ticker-shaped but resolvable nowhere	1	Friendly clarify card echoing the input, with example tickers and questions. No crash, no data fetch.

Repeating any of these queries within the hour costs **zero** AI calls — the full answer is served from the response cache (verified: second run made no provider calls at all).

6. Simulated Failure Scenarios and What the User Sees

Each failure below was injected deliberately (bad API keys, fake hung sockets, stripped data) and the user-facing behaviour recorded. The design principle: **graceful degradation** — a useful message, never a stack trace.

Failure injected	What the user sees	Why it is safe
Same question asked twice within an hour	Identical full answer, instantly.	Response cache keyed on the exact prompt; second run verified to hit zero providers. Free-tier quota is spent once per unique question per hour.
Groq (primary AI) down — invalid key / 401	A completely normal forecast. Nothing visible changed.	The router logged the failure and silently switched both calls to Gemini. Verified live: 401 from Groq, full report from Gemini, critic approved.
BOTH AI providers down	A calm error card: 'Couldn't complete the analysis for ITC... if it's a rate limit, wait a moment — results are cached, so a retry is cheap', plus the disclaimer.	Long provider error blobs are truncated; details stay in the logs. The app remains responsive.

Failure injected	What the user sees	Why it is safe
No annual reports ingested (RAG empty)	Normal answer; the agent trace notes '0 annual-report snippets'.	RAG is optional by design; retrieval returns an empty list rather than raising.
Yahoo returns partial data (dividends but no financial statements)	A full forecast, with the affected metrics shown as '—' and plain-language data notes explaining each gap (e.g. 'No cash-flow statement; coverage unavailable').	Every metric degrades independently to 'unavailable + explanation'; the forecast uses whatever is computable and the critic still validates it.
Yahoo Finance hangs (simulated stuck socket)	After the 45-second cap: 'Fetching ITC timed out — Yahoo Finance may be slow or unreachable. Please try again in a moment.'	The fetch runs under a wall-clock timeout in a worker thread; timeouts are never written to the cache, so the next attempt is a genuine retry.
Critic rejects, but the 3-call budget is spent	The answer ships with a LOW confidence badge, an extra risk line, and the critic's specific objections in the agent trace.	Honest degradation: the disagreement is shown, not hidden, and the quota ceiling holds (verified live: exactly 3 calls).
Invalid ticker (e.g. a typo that looks like a ticker)	Clarify card echoing what was tried, with valid examples.	The lookup table and a live existence check both have to fail before this path; the LLM is never allowed to guess.

7. Real Bugs the Testing Process Caught

The simulations were not theatre — they surfaced genuine production issues, several of which would have been embarrassing to discover during a demo:

- **The fallback provider was dead and nobody knew.** Google retired the pinned model (gemini-2.5-flash) overnight — it answered on July 9 and returned '404: no longer available' on July 10. Because Groq always answered first, no normal run would ever have noticed. The Groq-down simulation exposed it; the fix probes the live model list and pins a model that actually responds (gemini-3.1-flash-lite), with instructions in the config for the next retirement.
- **New Gemini models changed their answer format.** They return a list of text blocks instead of a plain string; the old code would have stringified Python objects into the JSON parser. Caught the moment the new model was tested; fixed with a content normaliser plus unit tests.
- **The critic retry loop silently doubled AI usage.** Two of three live question queries triggered a retry (5 calls instead of 3) for the same root cause: the forecast prompt never said how the range should relate to the growth metrics, so the critic kept objecting. Fixed twice over: the prompt now anchors the range on last year's dividend and the growth signals, and the hard budget makes an overrun impossible regardless.
- **Coal India looked like half the stock it is.** The half-collected fiscal year (final dividend not yet ex-dividend) understated the yield as 3.6% instead of 7.4% — caught by manually cross-checking the metrics table against Screener.in, fixed with provisional-year detection.
- **An unquoted comma broke a CSV.** The note field 'SAMPLE DATA — placeholder, not verified' split into a phantom eighth column. Caught because the verification script printed the parsed row and the note was visibly truncated.
- **Windows consoles cannot print the rupee sign.** Every CLI script needed a UTF-8 guard or the demo would crash on its own currency symbol. Caught on the first real CLI run.

8. How the System Is Tested

The test suite has **165 automated tests** in three layers, plus the live simulations described above:

- **Unit tests (fast, offline)** — every module's logic with fakes standing in for the network and the AI: chunking arithmetic, fiscal-year attribution, cache TTL expiry, quota-window math, JSON repair, every clarify/out-of-scope branch of the intent parser, and the full graph routing including the retry budget (a parameterised test asserts 'total AI calls is 3 or fewer' across every scenario).
- **Integration tests (live, marked)** — a handful of tests hit real Yahoo Finance for ITC and Coal India and assert the results are structurally complete and in plausible ranges. They are tagged so 'pytest -m "not integration"' runs the offline suite alone in about eight seconds.
- **UI tests** — Streamlit's AppTest framework renders every branch of the web page headlessly (forecast card, direct-answer banner, clarify/out-of-scope/error cards, and the streamed progress path with a scripted graph).
- **Live audits** — an instrumented harness runs the real pipeline against real providers, counts every provider call, and cross-checks every number the AI cites against the computed metrics.

9. What the Architecture Leaves Room For

The MVP deliberately keeps seams open. Each roadmap item plugs into an existing interface without touching the rest of the system:

Enhancement	Where it plugs in
Screener / ranking queries ('top PSU dividend payers')	A second branch in the graph; the intent parser already classifies these (currently as out-of-scope).
Live NSE corporate-actions scraper	A new class implementing the existing CorporateActionsSource interface; one line changes in the factory.
Conversation memory ('what about TCS?')	LangGraph checkpointing + session state.
Watchlist alerts, backtesting, peer comparison	New modules reusing the existing pipeline.
Local LLM via Ollama (zero quota)	A third provider entry in the router.
FastAPI + React, multi-user	Tier 4 swap; tiers 1–3 untouched.

10. Closing Note

The through-line of this architecture is a simple division of labour: **deterministic code wherever money is counted, AI wherever judgement is needed, and a checker between the two**. Everything else — the caches, the budgets, the fallbacks, the timeouts, the friendly error cards — exists to keep that core loop trustworthy on a laptop and a free tier. Every behaviour described in this document was executed and observed, and the full history is in the repository's commit log.